The most foundational difference between the languages is that Ruby is object-oriented while Elixir is function-oriented. Objects combine data and methods (functions) into a single instance, and objects interact with each other to form your application. Elixir has no objects, so functions must be provided the data that they operate on. You'll see Elixir patterns and concepts that feel a bit like objects, but don't be tricked—Elixir doesn't have objects.

Elixir has fewer features compared to Ruby. Because there are no objects, there is no inheritance. Additionally, data like integers, strings, and maps cannot be directly extended with helper methods. This can create the feeling of "how can I build the same app with fewer features at my disposal?" Don't worry, Elixir has everything you need to write great software.

Another big difference between Ruby and Elixir is data mutability. This refers to whether or not data can be directly modified after it's created. A simple demonstration of this is with a list of `[1, 2, 3]`. In Ruby, you can call `list.pop` and you'll receive the result `(3)`. But the underlying data in `list` changes to `[1, 2]`. In Elixir, you can call `List.pop_at(list, -1)` and you'll receive both the result and the new list. The data in `list` remains the same.

There isn't an objectively right answer to which of these designs is better. But many people, including myself, prefer Elixir's immutable data and functional nature because it increases predictability and readability.

These are just some of the major differences between the languages. There are many more, ranging from the typical program's structure to the way that code executes. You'll undoubtedly see and make comparisons between the two as you learn Elixir. This is a good thing because you'll more quickly grasp concepts, but don't try to turn Elixir into Ruby, or vice-versa.

With that out of the way, let's look more closely at Elixir! We'll start with data types.

Data Types

Elixir has a simple set of data types—the usual suspects are all here. You use these types to store your application data and pass it as arguments to functions.

Ruby differs a bit in what you can do with basic data types. Everything in Ruby (including basic data types) is an object, so you can add methods to the object class in order to extend the data type. For example, Ruby on Rails has implemented methods to make syntax like `10.days.from_now` possible. This works because ActiveSupport has extended the `Integer` class with these methods.